

# AI-Based Driver Behaviour, Fatigue and Risk Detection System

A Machine Learning Approach Using CNN and LSTM



Submitted by

**Debmalya Halder**

4th Year, ECE, Section-B

Roll No.: 140

Enrolment No.: 12023052003141

Department of Electronics and Communication Engineering  
Institute of Engineering and Management (IEM)  
Kolkata, West Bengal, India

# TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. System Requirements
6. Proposed Architecture
7. Input Parameters and Features
8. Data Collection and Pre-processing
9. CNN-Based Visual Analysis
10. LSTM-Based Temporal Modelling
11. Driver Behaviour Classification
12. Risk Score and Decision Logic
13. System Workflow
14. Implementation Plan
15. Evaluation Metrics
16. Expected Results
17. Safety, Privacy and Limitations
18. Future Enhancements
19. Conclusion
20. References

# 1. Abstract

Road safety depends not only on vehicle condition but also on driver behaviour, attention and fatigue. This project proposes an AI-based Driver Behaviour, Fatigue and Risk Detection System that combines camera-based perception with vehicle and motion sensor features. The system monitors speed, acceleration, braking, steering, lane deviation, eye closure, head position and phone usage. A convolutional neural network (CNN) extracts spatial features from images or video frames, while a long short-term memory (LSTM) model analyses sequences over time. The resulting representation is used to classify driver states such as normal driving, aggressive driving, distracted driving, drowsiness, unsafe braking and unsafe acceleration. A risk-scoring module combines the behaviour class with temporal indicators and vehicle dynamics to estimate the current level of driving risk. The proposed architecture is suitable for an Android/edge-computing prototype, an in-vehicle embedded system, or a cloud-assisted monitoring platform. The design emphasises low-latency inference, interpretable alerts and modular expansion for future datasets and deployment environments.

**Keywords:** driver monitoring, fatigue detection, drowsiness detection, distracted driving, CNN, LSTM, machine learning, risk scoring, computer vision, sensor fusion.

# 2. Introduction

Driver monitoring systems use sensing and machine learning to estimate whether a driver is attentive and operating a vehicle safely. The challenge is that risky behaviour is rarely represented by a single measurement. For example, a short increase in steering activity may be harmless, whereas repeated lane departures combined with eye closure and phone use can indicate substantially greater risk. This motivates a multimodal system in which visual features and vehicle dynamics are considered jointly.

The project therefore adopts two complementary AI ideas. CNN models are suited to extracting spatial patterns such as eyes, face orientation, lane regions and handheld-phone presence from images. LSTM networks are suited to modelling temporal dependencies such as repeated braking, acceleration bursts, prolonged eye closure and evolving lane deviation. The architecture combines these signals into behaviour classes and an overall risk score.

# 3. Problem Statement

Develop an AI system that continuously observes driver and vehicle signals and identifies potentially unsafe situations in near real time. The system should distinguish between normal driving and multiple unsafe behaviours, provide a risk score, and produce an alert when a threshold is exceeded. It should tolerate noisy sensor readings, missing frames and short-term fluctuations without generating excessive false alarms.

## Target classes

- Normal
- Aggressive
- Distracted
- Drowsy
- Unsafe braking
- Unsafe acceleration

# 4. Objectives

- Collect and organize camera/video and vehicle-sensor data.
- Extract visual features such as eye closure, head pose, lane position and phone presence.
- Model temporal behaviour using LSTM sequences.

- Fuse AI outputs with speed, acceleration, braking and steering indicators.
- Classify the driver state into predefined behaviour categories.
- Generate an interpretable risk score and alert state.
- Evaluate the model using classification and detection metrics.
- Design the system so that additional behaviours and sensors can be added later.

## 5. System Requirements

### 5.1 Hardware

- Camera: RGB dashboard/in-cabin camera, preferably 720p or higher.
- Vehicle or phone sensors: speed, accelerometer, gyroscope, steering/braking data where available.
- Compute: laptop/workstation for training; edge device or Android smartphone for prototype inference.
- Storage: sufficient local storage for labelled video clips, sensor logs and model checkpoints.

### 5.2 Software

- Python 3.x for data processing and model training.
- OpenCV for image/video processing.
- PyTorch or TensorFlow/Keras for CNN/LSTM training.
- NumPy/Pandas for data handling and feature preparation.
- Matplotlib/Seaborn or equivalent for analysis plots.
- Git/GitHub for version control and experiment tracking.

## 6. Proposed Architecture

The proposed architecture follows a modular pipeline. Camera and vehicle sensors first generate synchronized observations. A pre-processing stage cleans and normalizes the signals. The visual branch extracts frame-level features, and the temporal branch converts recent frame/sensor features into a behaviour representation. A fusion layer combines modalities before classification. Finally, a risk engine converts class probabilities and severity indicators into an alert level.

| Stage                    | Main function                                           |
|--------------------------|---------------------------------------------------------|
| Camera + Vehicle Sensors | Collect video frames and motion/vehicle signals         |
| Feature Extraction       | Face/eyes/head/lane/phone and driving-dynamics features |
| CNN / Visual Encoder     | Learn spatial visual representation                     |
| LSTM                     | Learn temporal behaviour patterns                       |
| Fusion + Classifier      | Combine modalities and estimate class probabilities     |
| Risk Score               | Map behaviour, probability and severity to risk level   |
| Alert / Dashboard        | Show warning, current class and trend                   |

High-level flow: **Camera + Sensors -> Pre-processing -> Feature Extraction -> CNN/LSTM -> Behaviour Class -> Risk Score -> Alert**

## 7. Input Parameters and Features

| Parameter    | Example feature                | Purpose                                      |
|--------------|--------------------------------|----------------------------------------------|
| Speed        | mean / max speed               | Detect speeding and context for harsh events |
| Acceleration | longitudinal acceleration      | Detect rapid acceleration or instability     |
| Braking      | deceleration / brake intensity | Identify harsh braking                       |
| Steering     | steering rate / angle variance | Detect aggressive or unstable control        |

| Parameter        | Example feature            | Purpose                               |
|------------------|----------------------------|---------------------------------------|
| Lane deviation   | distance from lane centre  | Detect drifting / inattention         |
| Eye closure      | PERCLOS / closure duration | Detect drowsiness                     |
| Head position    | yaw / pitch / roll         | Detect gaze diversion or head turning |
| Phone usage      | handheld phone probability | Detect distraction                    |
| Temporal history | last N seconds of features | Capture behaviour evolution           |

Derived features should be normalized so that sensor units do not dominate the learned representation. Sequence windows can be created using overlapping segments, for example several seconds of observations at a fixed sampling interval.

## 8. Data Collection and Pre-processing

A useful dataset should contain labelled examples of the target behaviours under different lighting conditions, road types, driver appearances, vehicle speeds and traffic situations. Each sample should be associated with a timestamp and, where possible, synchronized sensor data. Labels should be defined using a clear annotation guide to reduce ambiguity between normal and unsafe events.

### Pre-processing pipeline

- Resize video frames and normalize pixel values.
- Apply face/eye/head or pose detection as required by the selected vision model.
- Compute lane position or lane deviation from road-camera frames.
- Filter sensor noise using moving average, median filtering or a low-pass filter as appropriate.
- Synchronize video and sensor streams using timestamps.
- Create fixed-length sequences with labels assigned to the sequence endpoint or interval.
- Split data by driver/session rather than randomly by frame to reduce identity leakage.

Data augmentation can include changes in brightness, crop, small rotations, blur and horizontal variation where physically meaningful. For sensor sequences, limited noise injection and scaling augmentation may be used cautiously.

## 9. CNN-Based Visual Analysis

The CNN branch converts each frame into a compact feature vector. Depending on available compute, the project may use a lightweight architecture such as MobileNet-style or EfficientNet-style backbones, or a custom CNN. The visual branch can receive a cropped driver region or a full frame. Its outputs may represent eye status, face orientation, lane context and phone-use evidence.

### Conceptual computation:

```
Frame -> Resize/Normalize -> CNN Backbone -> Feature Vector -> Visual Probabilities / Embedding
```

For an edge deployment, model size, inference time, memory consumption and thermal limits should be measured in addition to accuracy. Quantization or model pruning can later be evaluated if necessary.

## 10. LSTM-Based Temporal Modelling

A single image may not reliably distinguish a blink from prolonged eye closure or a routine steering correction from unstable driving. Temporal modelling addresses this limitation. A sequence of feature vectors is fed into an LSTM, which maintains a hidden state that summarizes recent observations.

### Example sequence input:

```
x_t = [speed, acceleration, brake, steering, lane_dev, eye_closure, head_pose, phone_prob, visual_embedding]
```

The LSTM output can be passed to a dense classifier to estimate the probability of each behaviour class. A smoothing stage can then reduce rapid label flicker between adjacent frames.

## 11. Driver Behaviour Classification

| Class      | Representative evidence                                              |
|------------|----------------------------------------------------------------------|
| Normal     | Stable speed, centred lane, attentive eyes/head, no phone indication |
| Aggressive | Rapid acceleration, high steering variability, repeated harsh events |

| Class               | Representative evidence                                                   |
|---------------------|---------------------------------------------------------------------------|
| Distracted          | Phone-use evidence, gaze/head away from road, lane deviation              |
| Drowsy              | Prolonged or repeated eye closure, yawning/gaze degradation, head nodding |
| Unsafe braking      | Large braking/deceleration event or repeated harsh braking                |
| Unsafe acceleration | Large positive acceleration event or repeated rapid acceleration          |

Because behaviours can overlap, the model should support a primary class plus auxiliary indicators. For example, a sequence may be classified primarily as distracted while simultaneously increasing a fatigue-related indicator.

## 12. Risk Score and Decision Logic

The risk score is an engineering layer placed after model inference. It should be interpretable and calibrated on validation data. One practical design is a weighted sum of class probability, event severity and persistence.

```
Risk = w_c * ClassSeverity + w_p * Persistence + w_s * SensorSeverity + w_t * Trend
```

A three-level output can be used: Low, Medium and High. For example, a short mild deviation may remain low/medium, while persistent drowsiness, phone use with lane deviation, or severe vehicle dynamics can move the score to high. Exact weights and thresholds should be learned or validated empirically rather than hard-coded without testing.

## 13. System Workflow

- Step 1:** Start the driver-monitoring session.
- Step 2:** Capture camera frames and sensor data.
- Step 3:** Pre-process and synchronize all inputs.
- Step 4:** Extract visual and numerical features.
- Step 5:** Run CNN visual inference.
- Step 6:** Build a temporal feature sequence.
- Step 7:** Run LSTM temporal inference.
- Step 8:** Fuse outputs and calculate behaviour probabilities.
- Step 9:** Calculate and smooth the risk score.
- Step 10:** Trigger visual/audio/haptic alert when thresholds are met.
- Step 11:** Log events for evaluation and later improvement.

## 14. Implementation Plan

### 14.1 Module structure

- `data_ingestion`: reads videos and sensor logs.
- `preprocess`: normalization, synchronization and sequence creation.
- `vision_model`: CNN backbone and visual heads.
- `temporal_model`: LSTM encoder and sequence classifier.
- `fusion`: combines visual and sensor representations.
- `risk_engine`: class severity, persistence, smoothing and alert thresholds.
- `evaluation`: metrics, confusion matrix and error analysis.
- `deployment`: real-time inference service / Android or edge integration.

### 14.2 Pseudocode

```
while session_active:
    frame = camera.read()
    sensor = sensors.read()
    visual_features = cnn(preprocess_frame(frame))
    state = build_feature_vector(visual_features, sensor)
    sequence.append(state)
    if len(sequence) == WINDOW:
        probs = lstm(sequence)
        risk = risk_engine(probs, sequence)
        display_state(probs, risk)
    sequence.pop(0)
```

For a production-quality prototype, inference should run asynchronously so camera capture, sensor acquisition, inference and UI updates do not block each other. The system should also include fail-safe handling for invalid frames, sensor dropouts and model initialization errors.

## 15. Evaluation Metrics

The model should be evaluated using metrics that reflect class imbalance and the cost of missed unsafe events.

- **Accuracy**: proportion of correct class predictions.
- **Precision**: fraction of predicted positives that are correct.
- **Recall**: fraction of true positive events detected.
- **F1-score**: harmonic mean of precision and recall.
- **Confusion matrix**: shows which behaviour classes are confused.
- **ROC-AUC / PR-AUC**: useful for binary or one-vs-rest analyses.
- **False-alarm rate**: important for alert usability.
- **Latency**: end-to-end time from input capture to alert.
- **Frame rate / throughput**: confirms real-time feasibility.

Evaluation should be performed on held-out drivers or sessions. Driver-independent testing is important because a system can otherwise appear accurate by memorizing visual characteristics of the same participants seen during training.

## 16. Expected Results

The completed system is expected to produce a rolling estimate of driver state and a risk score while preserving low enough latency for interactive alerts. Expected qualitative outcomes include stable recognition of normal

driving, improved detection of prolonged eye closure compared with frame-only logic, improved robustness to short events through LSTM sequence modelling, and more informative alerts when visual and vehicle evidence agree.

A final academic evaluation should report the actual measured values obtained from the selected dataset and hardware. The report should not claim a target accuracy as a measured result until experiments are completed.

## **17. Safety, Privacy and Limitations**

### **17.1 Safety considerations**

- The system is an assistive monitoring tool, not a substitute for responsible driving.
- Alerts should be designed to reduce distraction rather than create additional distraction.
- A fail-safe state should be used when confidence is low or the camera/sensor is unavailable.
- Thresholds should be tested in controlled conditions before any real-road deployment.

### **17.2 Privacy considerations**

- Prefer local/edge processing for sensitive camera data when feasible.
- Store only the minimum data necessary for debugging and model improvement.
- Use access controls and encryption for recorded sessions.
- Separate research identifiers from personal identifiers in datasets.

### **17.3 Limitations**

Performance may degrade under poor illumination, occlusion, unusual camera placement, missing lane markings, sensor drift, extreme weather or behaviours that are under-represented in the training data. Driver behaviour can also be context dependent, so a single class label may not fully describe an event. These limitations should be documented through an error-analysis section and considered before deployment.

## 18. Future Enhancements

- Add yawning detection and blink-rate trends for stronger fatigue modelling.
- Use transformer-based temporal models for comparison with LSTM.
- Add vehicle CAN/OBD-II signals where legally and technically appropriate.
- Introduce personalized baseline adaptation while preserving privacy.
- Add uncertainty estimation and confidence-aware alerts.
- Optimize the model using quantization for mobile/edge devices.
- Build a dashboard for trip summaries, event timelines and risk trends.
- Create a federated-learning or privacy-preserving training option for multi-device datasets.

## 19. Conclusion

This project presents a complete conceptual design for an AI-based Driver Behaviour, Fatigue and Risk Detection System. The central idea is to combine visual understanding from CNNs, temporal modelling from LSTMs, and vehicle/sensor features inside a single decision pipeline. The system targets normal, aggressive, distracted, drowsy, unsafe braking and unsafe acceleration behaviours and converts the inferred state into a continuous risk indicator. The modular architecture makes it possible to begin with an academic prototype and progressively move toward an optimized edge deployment. Future experimentation should focus on driver-independent datasets, real-time latency, false-alarm control, calibration and careful validation under varied conditions.

## 20. References

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press.
- [2] F. Chollet, *Deep Learning with Python*, Manning.
- [3] C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer.
- [4] OpenCV documentation, computer vision and image processing reference.
- [5] PyTorch documentation, model training and deployment reference.
- [6] TensorFlow / Keras documentation, deep learning model development reference.
- [7] Research literature on driver drowsiness detection, distracted-driving detection, driver monitoring systems, and sequential modelling should be added here for the final academic submission, with exact dataset/model citations matching the experiments performed.

## Appendix A. Project Specification Used

The project specification supplied for this report defines the following parameters: Speed, Acceleration, Braking, Steering, Lane deviation, Eye closure, Head position and Phone usage. The requested system identifies normal driving, aggressive acceleration, sudden braking, distracted driving, drowsiness and unsafe driving. The proposed architecture is Camera + Vehicle Sensors -> Feature Extraction -> CNN/LSTM -> Driver Behaviour -> Risk Score, with possible classes Normal, Aggressive, Distracted, Drowsy, Unsafe braking and Unsafe acceleration.